

Dostęp do baz danych w technologii .NET

Accessing databases from .NET applications

dr hab. inż. Maciej Grzenda, prof. uczelni

Maciej.Grzenda@pw.edu.pl

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Objectives

- .NET is one of the most popular developer platforms
- Its predecessors are .NET Framework, which provided the basis for .NET Core i.e. an open-source cross-platform implementation available at <https://github.com/dotnet/core>, next renamed to .NET
- .NET is used to develop both desktop and web applications
- Hence, not surprisingly, .NET is also used for developing database applications
- The objective of this lecture is to discuss how connections to databases can be made and used for querying and transactional processing with .NET components

ADO.NET

- *ADO.NET provides consistent access to data sources such as SQL Server and XML, and to data sources exposed through OLE DB and ODBC. Data-sharing consumer applications can use ADO.NET to connect to these data sources and retrieve, handle, and update the data that they contain.*
- *ADO.NET separates data access from data manipulation into discrete components that can be used separately or in tandem. ADO.NET includes .NET Framework data providers for connecting to a database, executing commands, and retrieving results. (...)*
- *The ADO.NET classes are found in System.Data.dll, and are integrated with the XML classes found in System.Xml.dll.*

1. ADO is an abbreviation for ActiveX Data Objects
2. This material is cited from <https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-overview>
3. Note that ADO.NET provides low level access to databases.

Entity Framework Core

- A (partial) alternative to ADO.NET is Entity Framework Core (see <https://learn.microsoft.com/en-us/ef/core/> for details).
- It can serve as an object-relational mapper (O/RM).
- O/RM solutions address the challenge of mapping objects e.g. created in Java or C# to records in relational databases (and vice versa)
- With O/RM, a developer can work with objects of e.g. Customer class that will be synchronised with records in a database table. In the case of Entity Framework Core, this is done by an object of DbContext class.
- Note that O/RM frameworks can be used to create also objects (of e.g. Customer class) from records. This reduces the need to write trivial SELECT/INSERT/UPDATE/DELETE statements aimed at reading/inserting/updating/deleting a single record
- Similar O/RM solutions exist for other programming languages, e.g. for Java.

.NET Framework – key data providers

Provider	Provides access to the data of	Namespace used
.NET Framework Data Provider for SQL Server	Microsoft SQL Server	System.Data.SqlClient
.NET Framework Data Provider for OLE DB	Data sources exposed by using OLE DB.	System.Data.OleDb
.NET Framework Data Provider for ODBC	Data sources exposed by using ODBC.	System.Data.Odbc
.NET Framework Data Provider for Oracle	Oracle data sources.	System.Data.OracleClient

1. Both dedicated Data Providers and a generic data provider based providing an access to ODBC data sources are available.
2. Further details: <https://docs.microsoft.com/en-us/dotnet/framework/data/adonet/data-providers>
3. ODBC stands for Open Database Connectivity. See <https://learn.microsoft.com/en-us/cpp/data/odbc/odbc-basics?view=msvc-170> for details on this standard which is defined by Microsoft as the database portion of the Microsoft Windows Open Services Architecture (WOSA)

ACCESSING MS SQL SERVER DATABASES FROM .NET APPLICATIONS

Connections – an example

```
using System;
using System.Data.SqlClient;
using System.Data;
namespace SampleDatabaseApplication
{
    static class Program
    {
        ...
        static void Main() {
using (SqlConnection con = new SqlConnection()) {
            con.ConnectionString =
"Data Source=localhost\\START;Database=Northwind;Integrated
Security=false;User Id=myUserId;Password=myPassword";
            con.Open();
            if (con.State==ConnectionState.Open)
            {
                Console.WriteLine("Connection has been
established");
                DisplayEmployees(con);
            }
            else
            {
                ...
            }
        }
    }
}
```

A Connection to Ms SQL Server is established. Server and instance names have to be supplied. DisplayEmployess() method is discussed on a different slide. Note that using() statement plays a similar role as try-with-resources in Java

SQLConnection string – selected settings – part I

Keyword	Default	Description
Connect Timeout	15	The length of time (in seconds) to wait for a connection to the server before terminating the attempt and generating an error.
Data Source	N/A	The name or network address of the instance of SQL Server to which to connect. The port number can be specified after the server name: server=tcp:servername, portnumber
Encrypt	'false'	When true, SQL Server uses SSL encryption for all data sent between the client and server if the server has a certificate installed. Recognized values are true, false, yes, and no.
Failover Partner	N/A	The name of the failover partner server where database mirroring is configured.
Database	N/A	The name of the database.

This illustrates how client components (in this case .NET components) can be used to set up high availability DBMS services.

1. Among other settings, the network protocol used to communicate with the server can be requested.
2. Cited from and further settings available at <https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqlconnection.connectionstring>

SqlConnection string – selected settings – part II

Keyword	Default	Description
Integrated Security	'false'	When false, User ID and Password are specified in the connection. When true, the current Windows account credentials are used for authentication.
MultipleActiveResult Sets	'false'	When true, an application can maintain multiple active result sets (MARS). When false, an application must process or cancel all result sets from one batch before it can execute any other batch on that connection.
Network Library	N/A	The network library used to establish a connection to an instance of SQL Server.
Password -or- Pwd	N/A	The password for the SQL Server account logging on. Not recommended. To maintain a high level of security, we strongly recommend that you use the Integrated Security or Trusted_Connection keyword instead.
User ID	N/A	The SQL Server login account. Not recommended. (...)

A recommended setting is to have one result set open at a time i.e. to preserve DBMS resources by not keeping results of multiple SQL queries in a connection.

Supported values include:
named pipes, apple talk, shared memory, IPX/SPX, and TCP/IP. Key DBMS platforms enable connections over different network protocols.

1. Cited from and further settings available at <https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqlconnection.connectionstring>

SQL Server Connection Pooling (ADO.NET)

- ADO.NET uses connection pooling
- The following solutions have been applied:
 - When a connection is opened and no pool exists for its connection string, a pool is created for this connection string
 - Hence, each connection pool is associated with a distinct connection string
 - A pool can be configured to include a minimum number of connections
 - Connection pooling is enabled by default in ADO.NET
- As in the case of JDBC connection pools:
 - Connection pooling reduces the number of times that new connections with DBMS must be established and improves overall performance in turn
 - The pooler is the owner of physical connection(s) in a pool.

Further details: <https://docs.microsoft.com/en-us/dotnet/framework/data/adonet/sql-server-connection-pooling>

SQL SELECT statements

```
public static void DisplayEmployees(IDbConnection con)
{
    IDbCommand com = con.CreateCommand();
    com.CommandType = CommandType.Text;
    com.CommandText = "SELECT FirstName, LastName from
Employees";
    using (IDataReader reader = com.ExecuteReader())
    {
        int i = 1;
        while (reader.Read())
        {
            Console.WriteLine(i + ": " +
reader["FirstName"] +
                                " " + reader["LastName"]);
            i = i + 1;
        }
    }
}
```

An SQL SELECT statement is executed. The result set is available via the reader object. The ExecuteReader method can be used for executing any code that returns the set of rows.

Data modification

```
public static void UpdateCustomers(IDbConnection con)
{
    IDbCommand com = con.CreateCommand();
    com.CommandType = CommandType.Text;
    com.CommandText =
        "UPDATE Customers set Country='Poland' WHERE
        Country='France'";
    int modifiedRows = com.ExecuteNonQuery();
    Console.WriteLine(modifiedRows+" rows have been
    modified");
}
```

1. **An SQL UPDATE statement is executed. The number of modified rows is returned.**
2. **com.CommandText = "EXECUTE ModifyCustomers"; can be used to execute a stored procedure instead.**

Parameterised data modification

```
public static void UpdateCustomers(SqlConnection con)
{
    SqlCommand com = new SqlCommand("UPDATE Employees set
        LastName=@LName where EmployeeId=@id", con);
    SqlParameter p1 = new SqlParameter();
    p1.ParameterName = "@LName";
    p1.SqlDbType = SqlDbType.VarChar;
    p1.Value="Orwell";
    com.Parameters.Add(p1);

    SqlParameter p2 = new SqlParameter();
    p2.ParameterName = "@id";
    p2.SqlDbType = SqlDbType.Int;
    p2.Value=2;
    com.Parameters.Add(p2);

    int modifiedRows = com.ExecuteNonQuery();
    Console.WriteLine(modifiedRows+" have been modified");
}
```

1. Parameterised queries can also be executed.
2. This is extremely important when user-supplied values are used in the queries. Such queries may be risky (see JDBC slides for details of SQL code injection)

Further details on Ms SQL Server connections

- For details on classes used in the previous examples and a complete list of classes, see <https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient>
- The `System.Data.SqlClient` namespace includes a number of other classes related to permissions, data loading, handling errors and other aspects of interacting with Ms SQL Server

ACCESSING ORACLE DATABASES FROM .NET APPLICATIONS

Oracle and .NET

- Oracle Data Provider for .NET (ODP.NET) enables developers to exploit Oracle database functionality
- This includes support for advanced functionality such as:
 - database clusters deployed with Oracle Real Application Clusters
 - fast connection failover
- Extensive documentation clarifies how options such as:
 - Connection pooling
 - Distributed transactions
 - Secure Sockets Layers
 - Retrieving query results as XMLcan be used.

Further details on the development of Oracle client applications using .NET can be found at

<https://www.oracle.com/database/technologies/appdev/dotnet/odp.html>

Oracle client applications

- Prior to developing .NET database applications accessing Oracle database, Oracle components may have to be installed
- Sample tools include Oracle Developer Tools for Visual Studio available at <https://www.oracle.com/database/technologies/developer-tools/visual-studio/>
- These tools make it possible to browse Oracle schema, compare schemas, generate scripts modifying target schema to make it match the source one, and generate .NET code when possible, or even stepping from .NET code into Oracle stored procedure code
- This illustrates how much the database layer and its code is linked to application development
- To run Oracle database applications, Oracle.DataAccess components may have to be added to Visual Studio project

Oracle connections

```
using System;
using System.Data;
using Oracle.DataAccess.Client;

namespace SampleDatabaseApplication
{
    static class Program
    {
        static void Main()
        {
            String oradb = "Data Source=(DESCRIPTION=" +
                "(ADDRESS_LIST=(ADDRESS=(PROTOCOL=TCP) (HOST=localhost) (PORT=1521))) "
                + "(CONNECT_DATA=(SERVER=DEDICATED) (SERVICE_NAME=ORCL2)))";
            + "User Id=hr;Password=hr;";
            OracleConnection conn = new OracleConnection(oradb);

            conn.Open();

            DisplayEmployees(conn);
            conn.Dispose();
        }
    }
}
```

A connection is set up. Connection string is used as in the case of tnsnames.ora file i.e. the primary Oracle configuration file. Instance name, TCP/IP port and security credentials are specified.

Oracle – sample result set

```
public static void DisplayEmployees(IDbConnection con)
{
    IDbCommand com = con.CreateCommand();
    com.CommandType = CommandType.Text;
    com.CommandText = "SELECT First_Name, Last_Name from
Employees";
    using (IDataReader reader = com.ExecuteReader())
    {
        int i = 0;
        while (reader.Read())
        {
            Console.WriteLine(i + ": " +
                reader["First_Name"] + " "
                + reader["Last_Name"]);
            i = i + 1;
        }
    }
    com.Dispose()
}
```

1. The same interface as for Ms SQL Server connections is used. Thus, only column names have to be changed to match HR.Employees column names.
2. OracleCommand can be used with Oracle connections
3. Pooling-related settings are also available.

Summary

- .NET platform, similarly to all leading platforms, provides database-related classes
- The capabilities are largely similar to JDBC capabilities:
 - A connection can be set up
 - Queries can be executed
 - Transactional processing is supported
- Not surprisingly, the APIs are similar in both cases.
- This is because they rely on the same underlying network interfaces and mechanisms of DBMS services. Also, the same threats are present.

Questions?

I am looking forward to your questions during the lectures (including now) and during my office hours

For details on my office hours, please check my website:

https://pages.mini.pw.edu.pl/~grzendam/eng/dydaktyka_eng.html





**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.